Grundlagen der Informationsvisualisierung

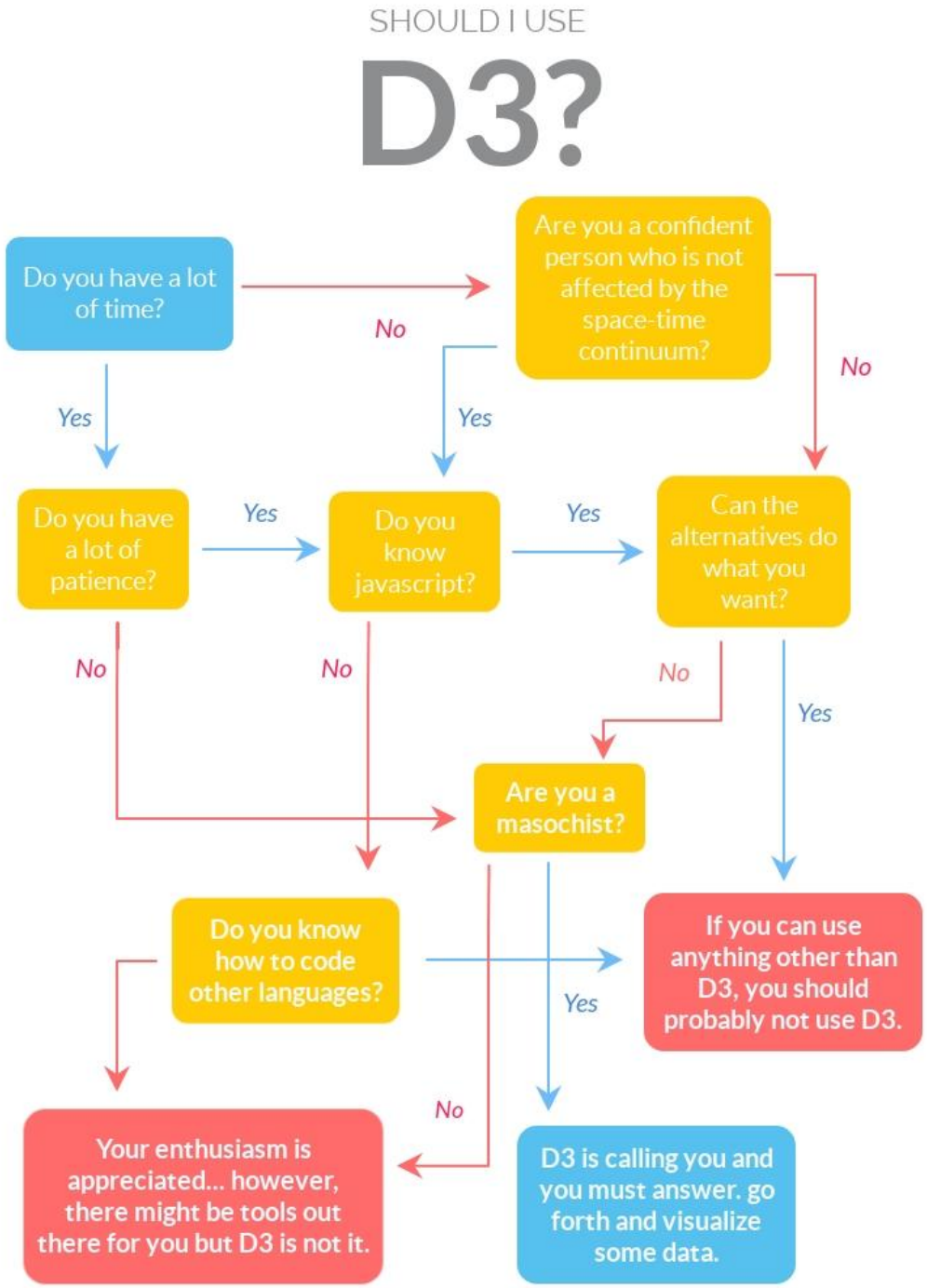
Exercise Session:
Week 2

Madhav Poddar,
Leah Mühlöder
Prof. Dr. Fabian Beck

Summer Semester
2026

University of Bamberg

D3.js



https://blog.planetargon.com/blog/entries/diving-into-d3

https://d3js.org/

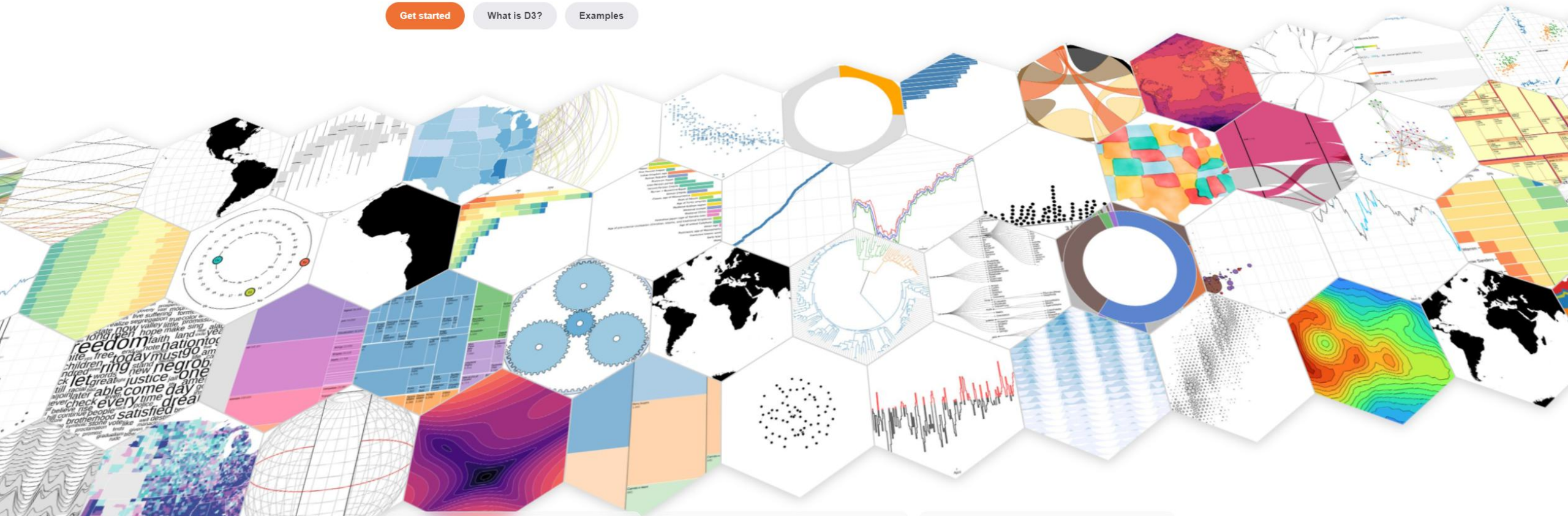
by Observable

Search Ctrl K

Home Examples Community Plot

Create custom dynamic visualizations
with unparalleled flexibility

Get started What is D3? Examples



Selections and transitions

Create, update, and animate the DOM based on data without the overhead of a virtual DOM.

Scales and axes

Encode abstract data into visual values such as position, size, and color. Explain position encodings with axes.

Shapes

Render arcs, areas, curves, lines, links, pies, stacks, symbols… and any geometric primitive you might need to visualize data.

Interactions

Layouts

Geographic maps

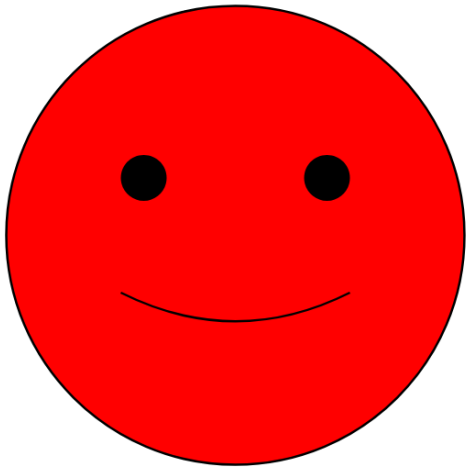
Repetition

- HTML: provides structure
- SVG: provides shapes
- JS: e.g. for selecting DOM-elements or changing attributes

D3 replaces neither HTML nor SVG. It helps us select DOM elements, bind data to them, and set SVG attributes more systematically.

D3.js in 100 seconds: https://www.youtube.com/watch?v=bp2GF8XcJdY

01: Select one element



Last week, we used JS:

```
…rest of the emojiD3-start.html code

<script>

const face = document.getElementById("face");

face.setAttribute("fill", "red");

</script>
```

This week, we use D3:

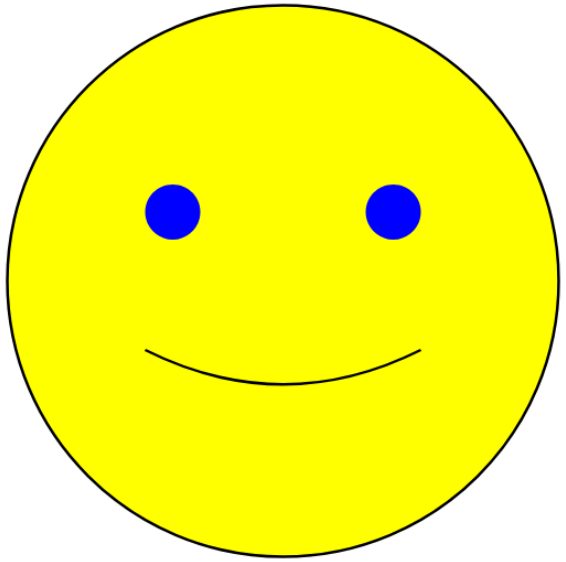
```
To use D3, load the library at the top of your HTML:
<script src="https://d3js.org/d3.v7.min.js"></script>
```

```
…rest of the emojiD3-start.html code

<script>
        d3.select("#face")
            .attr("fill", "red");

</script>
```

Task: Use emojiD3-start.html as starting point

1. Use D3.js to change the color of the face to a different color of your choice.
2. Add a thicker border.

02: select multiple elements



```
…added class “eye” to select svg-eyes via D3

<!-- Eyes -->
<circle class="eye" cx="170" cy="200" r="20" fill="black" />
<circle class="eye" cx="330" cy="200" r="20" fill="black" />
```

<u>Important:</u>

`select()` picks the first matching element, `selectAll()` picks all matching elements at once.

```
…rest of the emojiD3-start.html code

</script>
        d3.selectAll(".eye")
            .attr("fill", "blue");

</script>
```

Task:

1. Paint the eyes green.
2. Increase the size of the eyes slightly.
3. What happens when you use `select(".eye")`?

03 Bind data to exisiting elements

```
…rest of the emojiD3-start.html code

<script>

const eyeSizes = [20, 30]

            d3.selectAll(".eye")
                .data(eyeSizes)
                .attr("r", d => d);
</script>
```

```
…added class “eye” to select svg-eyes via D3

<!-- Eyes -->
<circle class="eye" cx="170" cy="200" r="20" fill="black" />
<circle class="eye" cx="330" cy="200" r="20" fill="black" />
```

03 Bind data to exisiting elements

```
…rest of the emojiD3-start.html code

<script>

const eyeSizes = [20, 30]

            d3.selectAll(".eye")
                .data(eyeSizes)
                .attr("r", d => d);
</script>
```

`attr()` can be assigned

- a value `.attr("r", 10)`
- a function `.attr("r", d => d)`

```
…added class “eye” to select svg-eyes via D3

<!-- Eyes -->
<circle class="eye" cx="170" cy="200" r="20" fill="black" />
<circle class="eye" cx="330" cy="200" r="20" fill="black" />
```

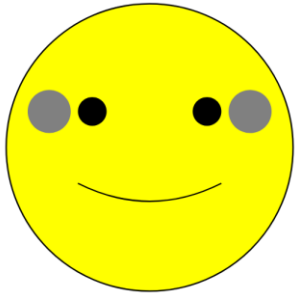
Task: Using a dataset, assign different colors to the eyes (while setting the size of the eyes to a fixed value).

04 Create new elements based on data

```
…rest of the emojiD3-start.html code

<script>

const data_eye_positions_x = [110, 390];


d3.select("#emoji_svg")
  .selectAll(".extra-eye")
  .data(data_eye_positions_x)
  .enter()
  .append("circle")
  .attr("class", "extra-eye")
  .attr("cx", d => d)
  .attr("cy", 200)
  .attr("r", 30)
  .attr("fill", "gray");
</script>
```

What does this code display in the browser?

04 Create new elements based on data



```
…rest of the emojiD3-start.html code

<script>

const eyeXPositions = [110, 390];     // ← the data: two x-positions


d3.select("#emoji_svg")               // ← pick the container (the SVG)
  .selectAll(".extra-eye")            // ← target elements → none exist yet!
  .data(eyeXPositions)                // ← bind the data array
  .enter()                            // ← one placeholder per missing element (2 in this case)
  .append("circle")                   // ← create a <circle> for each placeholder
  .attr("class", "extra-eye")         // ← tag it (so we can find it again)
  .attr("cx", d => d)                 // ← x comes from data (110, then 390)
  .attr("cy", 200)                    // ← same y for all
  .attr("r", 30)                      // ← same radius
  .attr("fill", "gray");              // ← same color
</script>
```

The D3 trick:

- `selectAll(".extra-eye")` finds **no** elements at first — and that's exactly the point.
- Together, `data + enter` tell D3: "Here's my data. For each item that doesn't have an element yet, remember a placeholder."
- Then `append` turns those placeholders into real circles.

Logic chain

Step	What it does
`.select`	pick the container (usually the `<svg>`)
`.selectAll`	select all target elements — *often empty at the start, that's intended*
`.data`	bind the data array to the selection
`.enter`	find data entries that don't have an element yet → placeholders
`.append`	create one new element per placeholder
`.attr`	set attributes (optionally based on the bound data)

In D3, we no longer describe each element individually—instead, we say:
<u>Create a visual object for each data point.</u>

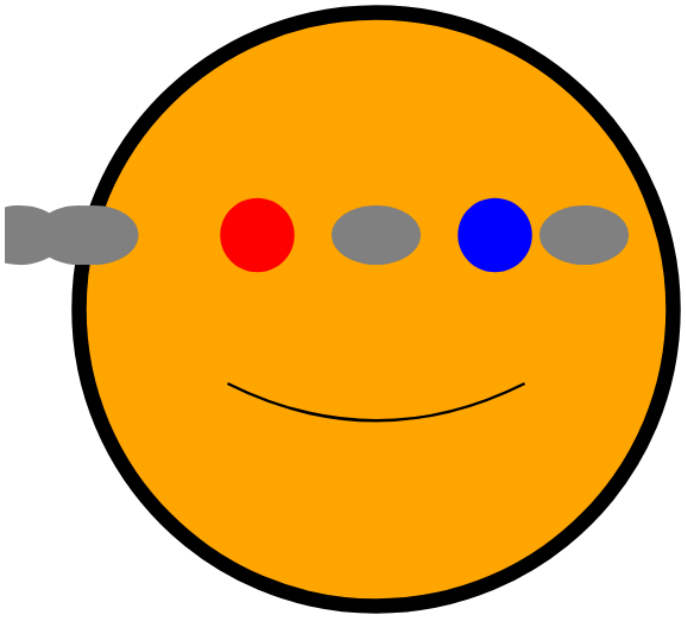
03 Create new elements based on data

```
…rest of the emojiD3-start.html code

<script>

const data_eye_positions_x = [110, 390];


d3.select("#emoji_svg")
  .selectAll(".extra-eye")
  .data(data_eye_positions_x)
  .enter()
  .append("circle")
  .attr("class", "extra-eye")
  .attr("cx", d => d)
  .attr("cy", 200)
  .attr("r", 30)
  .attr("fill", "gray");
</script>
```

Task:

1. Add 3 more extra eyes to the smiley face.
2. The extra eyes should be shaped like ellipses.

05 Scales



What you just experienced:

- circles cluster
- circles leave the SVG
- data is rarely in pixel units

→ Data does not automatically correspond to our screens.
→ **We need a translation from data value to pixel position.**
→ This is what a scale does.

05 Scales

```
const scale = d3.scaleLinear()
    .domain([0, 100])   // Daten: 0 bis 100
    .range([0, 300]);   // Pixel: 0 bis 300


scale(50)  // → 150
scale(100) // → 300
```

→ Scales are functions that map values from input (**domain**) to output (**range**).

Type	**Input → Output**	**When to use**
`scaleLinear`	numbers → numbers	continuous data (value → pixel)
`scaleOrdinal`	categories → discrete values	category → color
`scaleTime`	dates → numbers	time on an axis

d3js.org/d3-scale

Let‘s create our first bar chart

```
 <script>

const data = [10, 30, 50, 100];


const scaleDown = d3.scaleLinear()
    .domain([0, 100])
    .range([0, 300]);

d3.select("#barchartsvg")
    .selectAll("rect")
    .data(data)
    .enter()
    .append("rect")
    .attr("x", (d, i) => i * 40)
    .attr("y", 0)
    .attr("width", 30)
    .attr("height", d => scaleDown(d))
    .attr("fill", "steelblue");



</script>
```

What does this code display in the browser?

Let‘s create our first bar chart

```
<script>

const data = [10, 30, 50, 100];


const scaleDown = d3.scaleLinear()
    .domain([0, 100])
    .range([0, 300]);

d3.select("#barchartsvg")
    .selectAll("rect")
    .data(data)
    .enter()
    .append("rect")
    .attr("x", (d, i) => i * 40)
    .attr("y", 0)
    .attr("width", 30)
    .attr("height", d => scaleDown(d))
    .attr("fill", "steelblue");



</script>
```

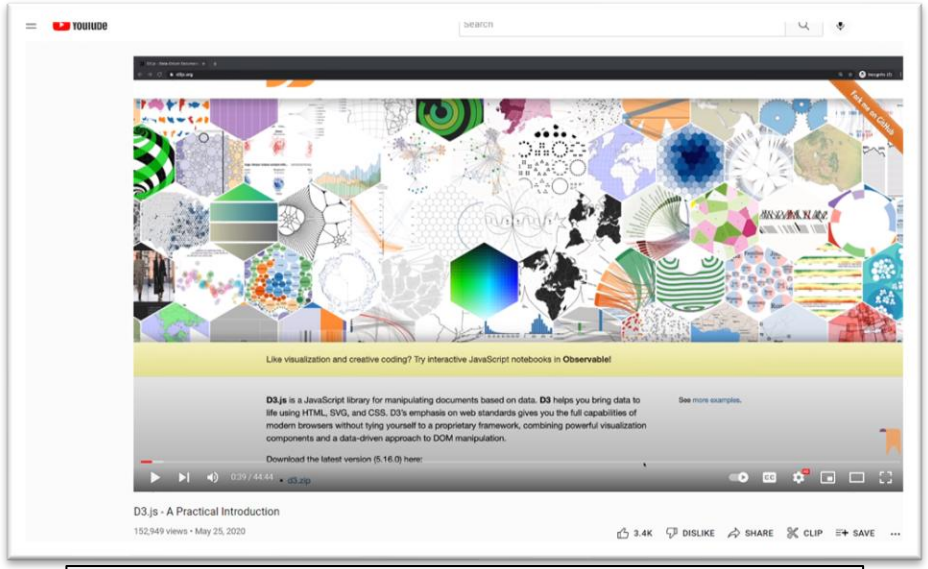
```
.attr(„x", (d, i) => i * 40)
```

- `"x"` — which attribute to set
- `(d, i)` — what D3 passes in: data value & index
- `i * 40` — return value = new x value

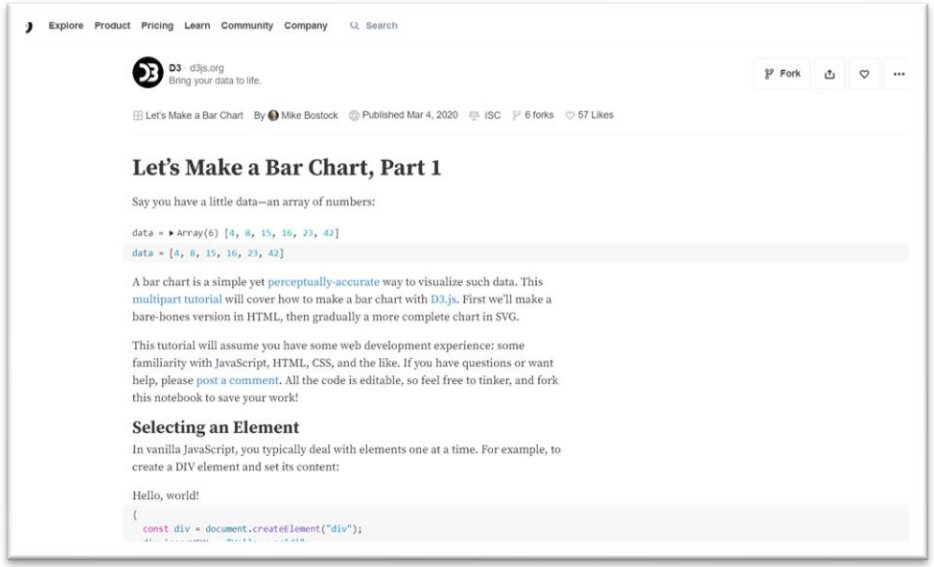
Task:

1. Modify the code so that the bars grow **vertically from the bottom upward** (like a classic bar chart you'd see in a newspaper)
2. Modify the code so that the bars grow **horizontally from left to right**, stacked from top to bottom.

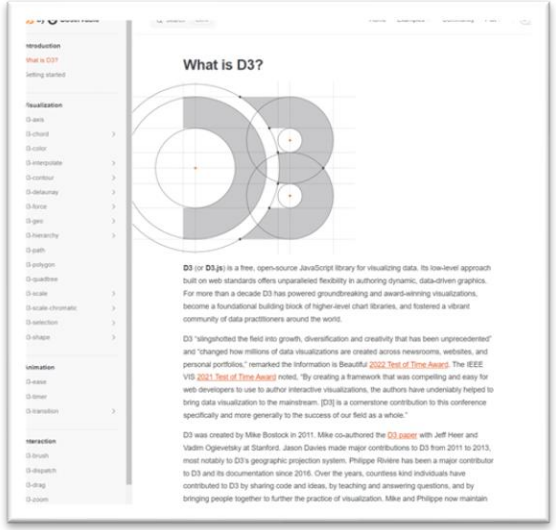
Tutorials



https://www.youtube.com/watch?v=TOJ9yjvlapY



https://observablehq.com/@d3/lets-make-a-bar-chart



https://d3js.org/what-is-d3